



Going Mainstream

Rick O'Connor

rickoco@riscv.org

<http://www.riscv.org>



Outline

- RISC-V ISA Background
- RISC-V Foundation Overview
 - Foundation Structure
 - Membership Update
- SW & HW Ecosystem
- Summary



Why Instruction Set Architectures Matter

- Why are 99%+ of laptops/desktops/servers based on AMD x86-64 ISA (over 95%+ built by Intel)?
- Why are 99%+ of mobile phones + tablets based on ARM v7/v8 ISA?
- Why can't Intel sell mobile chips and why can't ARM vendors sell servers?
- Because... the ISA is most important interface in a computer system
 - Where software meets hardware

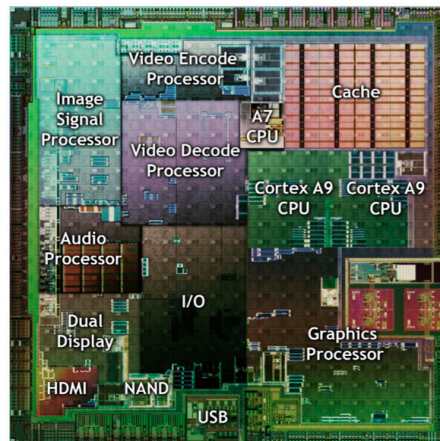
Open Software / Standards Work!

<i>Field</i>	<i>Standard</i>	<i>Free, Open Impl.</i>	<i>Proprietary Impl.</i>
Networking	Ethernet, TCP/IP	Many	Many
OS	Posix	Linux, FreeBSD	M/S Windows
Compilers	C	gcc, LLVM	Intel icc, ARMcc
Databases	SQL	MySQL, PostgreSQL	Oracle 12C, M/S DB2
Graphics	OpenGL	Mesa3D	M/S DirectX
ISA	??????	--	x86, ARM

- Why not a successful free & open ISA standard with open as well as proprietary implementations, like other fields?

Most CPU chips are SoCs with many ISAs

- Applications processor (usually ARM)
- Graphics processors
- Image processors
- Radio DSPs
- Audio DSPs
- Security processors
- Power-management processor
-



NVIDIA Tegra SoC

- Apps processor ISA too large for base accelerator ISA
- IP bought from different places, each proprietary ISA
- Home-grown ISA cores
- Over a dozen ISAs on some SoCs – each with unique software stack

So many ISAs!

Do we need all these different ISAs?

Must they be proprietary?

*What if there was one free and open
ISA everyone could use for everything?*

Benefits of Free Universal ISA

- Lower hardware development cost for all systems
- Lower software maintenance cost for all systems
- Greater innovation, through competition between open-source and proprietary implementations
- Computer architecture research becomes more relevant, directly applicable
- Enables OS/systems researchers to contemplate new hardware features

Universal ISA Requirements

- Works well with existing software stacks, languages
- Is native hardware ISA, not virtual machine/ANDF
- Suits all sizes of processor, from smallest microcontroller to largest supercomputer
- Suits all implementation technologies, FPGA, ASIC, full-custom, future device technologies...
- Efficient for all microarchitecture styles: in-order, decoupled, out-of-order; sequential, superscalar
- Supports extensive specialization to act as base for customized accelerators
- Stable: not changing, not disappearing

Why Didn't Other Open ISAs Take Off?

- SPARC V8 - To its credit, Sun Microsystems made SPARC V8 an IEEE standard in 1994
 - Sun, Gaisler offered open-source cores
 - ISA now owned by Oracle
- OpenRISC - GNU open-source effort started in 1999, based on DLX from Computer Architecture: AQA
 - 64-bit ISA was in progress in 2010
 - Didn't separate Architecture and Implementation
- Don't meet the needs of a free and open universal ISA



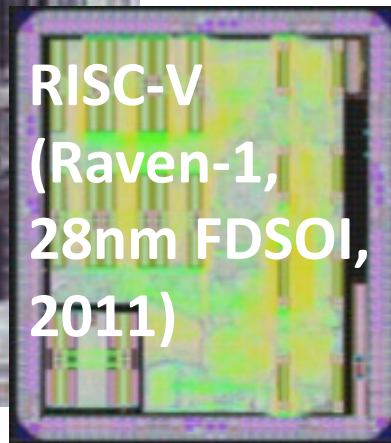
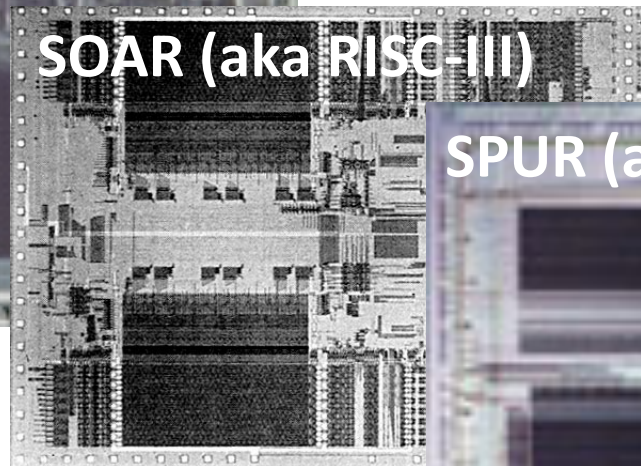
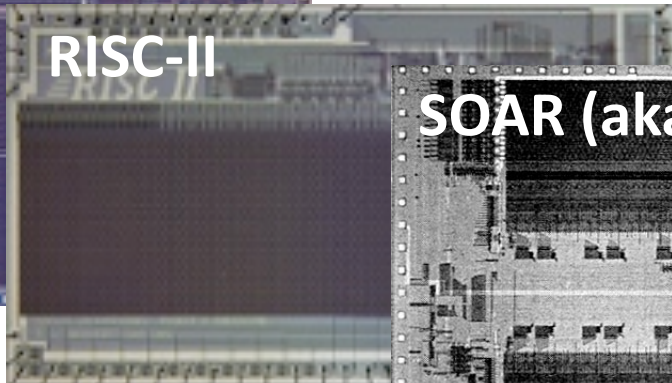
RISC-V Background

- In 2010, after many years and many projects using MIPS, SPARC, and x86 as basis of research, it was time for the Computer Science team at UC Berkeley to look at what ISAs to use for their next set of projects
- Obvious choices: x86 and ARM
 - x86 impossible – too complex, IP issues
 - ARM mostly impossible – complex, IP issues
- So UC Berkeley started “3-month project” during the summer of 2010 to develop their own clean-slate ISA

RISC-V Background (cont'd)

- Four years later, in May of 2014, UC Berkeley released frozen base user spec
 - many tapeouts and several research publications along the way
- The name RISC-V (pronounced *risk-five*), was chosen to represent the fifth major RISC ISA design effort at UC Berkeley
 - RISC-I, RISC-II, SOAR, and SPUR were the first four with the original RISC-I publications dating back to 1981
- In August 2015, articles of incorporation were filed to create a non-profit RISC-V Foundation to govern the ISA

Why the name RISC-V? (pronounced “risk-five”)



What's Different about RISC-V?

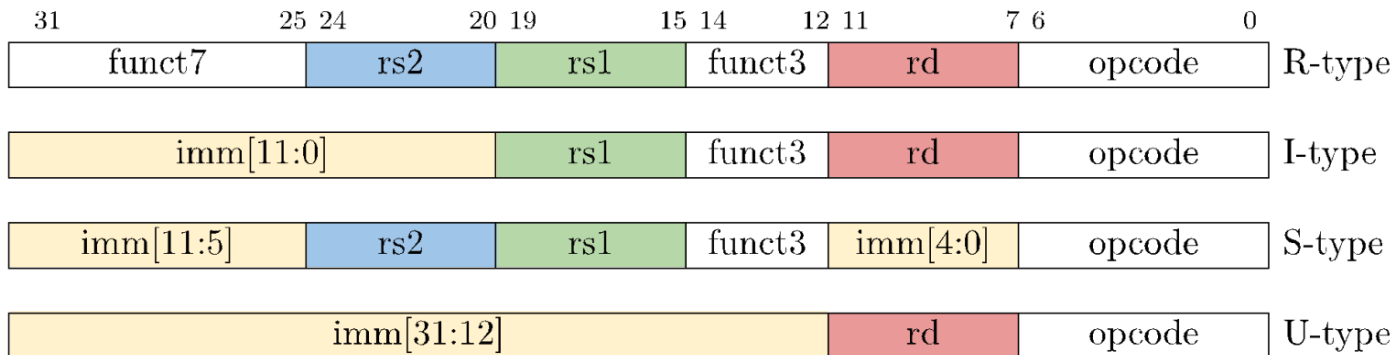
- **Simple**
 - Far smaller than other commercial ISAs
- **Clean-slate design**
 - Clear separation between user and privileged ISA
 - Avoids microarchitecture or technology-dependent features
- A **modular** ISA
 - Small standard base ISA
 - Multiple standard extensions
- Designed for **extensibility/specialization**
 - Variable-length instruction encoding
 - Vast opcode space available for instruction-set extensions
- **Stable**
 - Base and standard extensions are frozen
 - Additions via optional extensions, not new versions



RISC-V Base Plus Standard Extensions

- Four base integer ISAs
 - RV32C, RV32I, RV64I, RV128I
 - RV32C is 16-register subset of RV32I
 - Only <50 hardware instructions needed for base
- Standard extensions
 - M: Integer multiply/divide
 - A: Atomic memory operations (AMOs + LR/SC)
 - F: Single-precision floating-point
 - D: Double-precision floating-point
 - G = IMAFD, “General-purpose” ISA
 - Q: Quad-precision floating-point
- All the above are a fairly standard RISC encoding in a fixed 32-bit instruction format
- Above user-level ISA components frozen in 2014
 - Supported forever after

RISC-V Standard Base ISA Details



- 32-bit fixed-width, naturally aligned instructions
- 31 integer registers x1-x31, plus x0 zero register
- **rd/rs1/rs2** in fixed location, no implicit registers
- Immediate field (instr[31]) always sign-extended
- Floating-point adds f0-f31 registers plus FP CSR, also fused mul-add four-register format
- Designed to support PIC and dynamic linking



RV32I



②

③

RISC-V Reference Card ④

Base Integer Instructions (32[64]128)				
Category	Name	Fmt	RV(32[64]128) Base	
Loads	Load Byte	I	LB	rd,rs1,imm
	Load Halfword	I	LH	rd,rs1,imm
	Load Word	I	LW	rd,rs1,imm
	Load Byte Unsigned	I	LBU	rd,rs1,imm
	Load Half Unsigned	I	LHU	rd,rs1,imm
Stores	Store Byte	S	SB	rs1,rs2,imm
	Store Halfword	S	SH	rs1,rs2,imm
	Store Word	S	SW	rs1,rs2,imm
Shifts	Shift Left	R	SLL	rd,rs1,rs2
	Shift Left Immediate	I	SLLI	rd,rs1,shamt
	Shift Right	R	SRL	rd,rs1,rs2
	Shift Right Immediate	I	SRLI	rd,rs1,shamt
	Shift Right Arithmetic	R	SRA	rd,rs1,rs2
	Shift Right Arith Imm	I	SRAI	rd,rs1,shamt
Arithmetic	ADD	R	ADD	rd,rs1,rs2
	ADD Immediate	I	ADDI	rd,rs1,imm
	SUBtract	R	SUB	rd,rs1,rs2
	Load Upper Imm	U	LUI	rd,imm
	Add Upper Imm to PC	U	AUIPC	rd,imm
Logical	XOR	R	XOR	rd,rs1,rs2
	XOR Immediate	I	XORI	rd,rs1,imm
	OR	R	OR	rd,rs1,rs2
	OR Immediate	I	ORI	rd,rs1,imm
	AND	R	AND	rd,rs1,rs2
	AND Immediate	I	ANDI	rd,rs1,imm
Compare	Set <	R	SLT	rd,rs1,rs2
	Set < Immediate	I	SLTI	rd,rs1,imm
	Set < Unsigned	R	SLTU	rd,rs1,rs2
	Set < Imm Unsigned	I	SLTIU	rd,rs1,imm
Branches	Branch =	SB	BEQ	rs1,rs2,imm
	Branch ≠	SB	BNE	rs1,rs2,imm
	Branch <	SB	BLT	rs1,rs2,imm
	Branch ≥	SB	BGE	rs1,rs2,imm
	Branch < Unsigned	SB	BLTU	rs1,rs2,imm
	Branch ≥ Unsigned	SB	BGEU	rs1,rs2,imm
Jump & Link	J&L	UJ	JAL	rd,imm
	Jump & Link Register	I	JALR	rd,rs1,imm
Synch	Synch thread	I	FENCE	
	Synch Instr & Data	I	FENCE.I	
System	System CALL	I	SCALL	
	System BREAK	I	SBREAK	
Counters	Read CYCLE	I	RDYCLE	rd
	Read CYCLE upper Half	I	RDYCLEH	rd
	Read TIME	I	RDTIME	rd
	Read TIME upper Half	I	RDTIMEH	rd
	Read INSTR RETired	I	RDINSTRET	rd
	Read INSTR upper Half	I	RDINSTRETH	rd

+14
Privileged

+ 8 for M

+ 34
for F, D, Q

+ 46 for C

+ 11 for A

32-bit Instruction Formats

R	31	30	25	24	21	20	19	15	14	12	11	8	7	6	0
I	funct7		rs2		rs1		funct3		rd		opcode				
S	imm[11:0]		rs2		rs1		funct3		rd		opcode				
B	imm[11:5]		rs2		rs1		funct3		imm[4:0]		opcode				
U	imm[12]		imm[10:0]		rs2		rs1		funct3		imm[4:1]		imm[11]		
J	imm[20]		imm[10:3]		imm[31:12]		imm[11]		imm[19:12]		rd		opcode		



④

64{F|D|Q}/
128{F|D|Q}

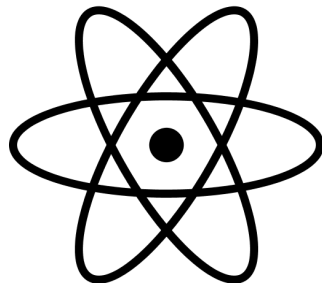


RISC-V Reference Card ④

Base Integer Instructions (32[64]128)				RV Privileged Instructions (32[64]128)				3 Optional FP Extensions: RV32{FIDQ}				Optional Compressed Instructions: RVC																																																																																																																																																																																																																																																																																																														
Category	Name	Fmt	RV32[64]128J Base	Category	Name	Fmt	RV mnemonic	Category	Name	Fmt	RV(FIDQ) (HP/SP,DP,QP)	Category	Name	Fmt	RVC																																																																																																																																																																																																																																																																																																											
Loads	Load Byte	I	LB rd,rsl,imm	CSR Access	Atomic R/W	R	CSRWR rd,csr,rsl	Load	Load	I	FLW(D,Q) rd,rsl,imm	Loads	Load Word	C	CL.W rd',rsl',imm																																																																																																																																																																																																																																																																																																											
	Load Halfword	I	LH rd,rsl,imm		Atomic Read & Set Bit	R	CSRRS rd,csr,rsl		Store	Store	S		FSW(D,Q) rsl,rsl,imm	Load Word SP	CI	C.LWSP rd,imm																																																																																																																																																																																																																																																																																																										
	Load Word	I	LW[D]Q rd,rsl,imm		Atomic Read & Clear Bit	R	CSRRC rd,csr,rsl			Arithmetic	ADD		R	FADD(S[D]Q) rd,rsl,rsl	Load Double	CL	C.LD rd',rsl',imm																																																																																																																																																																																																																																																																																																									
	Load Byte Unsigned	I	LBUR rd,rsl,imm		Atomic R/W Imm	R	CSRWWI rd,csr,imm				SUBTRACT		Subtract	R	FSUB(S[D]Q) rd,rsl,rsl	Load Double SP	CI	C.LWSP rd,imm																																																																																																																																																																																																																																																																																																								
	Load Half Unsigned	I	LH[W]DU rd,rsl,imm		Atomic Read & Set Bit Imm	R	CSRRSI rd,csr,imm						Multiply	FMUL	R	FMUL(S[D]Q) rd,rsl,rsl	Load Quad	CL	C.LQ rd',rsl',imm																																																																																																																																																																																																																																																																																																							
Stores	Store Byte	S	SB rsl,rsl,rsl,imm	Atomic Read & Clear Bit Imm	R	CSRRCI rd,csr,imm	Divide	FDIV				R		FDIV(S[D]Q) rd,rsl,rsl	Load Quad SP	CI	C.LQSP rd,imm																																																																																																																																																																																																																																																																																																									
	Store Halfword	S	SH rsl,rsl,rsl,imm	Change Level	Env. Call	R		ECALL	Square Root			FSQRT		R	FSQRT(S[D]Q) rd,rsl	Load Byte Unsigned	CL	C.LBU rd',rsl',imm																																																																																																																																																																																																																																																																																																								
	Store Word	S	SW[D]Q rsl,rsl,rsl,imm		Environment Breakpoint	R		EBREAK		Mul-Add		FMADD		R	FMADD(S[D]Q) rd,rsl,rsl,rsl	Float Load Word	CL	C.FLW rd',rsl',imm																																																																																																																																																																																																																																																																																																								
	Shifts	Shift Left	R		SLLI{W[D] rd,rsl,rsl	Environment Return		R			ERET	Negative Multiply-SUBTRACT		FMNSUB	R	FMNSUB(S[D]Q) rd,rsl,rsl,rsl	Float Load Double	CL	C.FLD rd',rsl',imm																																																																																																																																																																																																																																																																																																							
		Shift Left Immediate	R		SLLI{W[D] rd,rsl,rsl,shamt	Trap Redirect to Supervisor		Redirect Trap to Hypervisor			R		MRTS	Negative Multiply-ADD	FMNADD	R	FMNADD(S[D]Q) rd,rsl,rsl,rsl	Float Load Word SP	CI	C.FLWSP rd,imm																																																																																																																																																																																																																																																																																																						
Shift Right		R	SRLI{W[D] rd,rsl,rsl		Hypervisor Trap to Supervisor		R	MRTS			Sign Inject		FSGNJ		R	FSGNJ(S[D]Q) rd,rsl,rsl	Stores	Store Word	CS	C.SW rsl',rsl',imm																																																																																																																																																																																																																																																																																																						
Shift Right Immediate		R	SRLI{W[D] rd,rsl,rsl,shamt	Interrupt Wait for Interrupt	MMU Supervisor FENCE		R	SFENCE.VM rsl	Xor SIGN source				FSGNJN		R	FSGNJN(S[D]Q) rd,rsl,rsl		Store Word SP	CSS	C.SWSP rsl,imm																																																																																																																																																																																																																																																																																																						
Shift Right Arithmetic		R	SRAI{W[D] rd,rsl,rsl		Optional Multiply-Divide Extension: RV32M		Category	Name		Fmt			RV32M (Multi-Div)		Min/Max	Minimum		R	FMIN(S[D]Q) rd,rsl,rsl	Store Double	C	C.SD rsl',rsl',imm																																																																																																																																																																																																																																																																																																				
Shift Right Arithmetic Imm	R	SRAI{W[D] rd,rsl,rsl,shamt	Multiply									Multiply									R	MUL{W[D] rd,rsl,rsl,rsl	Compare Float	R	FEQ(S[D]Q) rd,rsl,rsl	Store Quad	CSS	C.SDSP rsl,imm																																																																																																																																																																																																																																																																																														
Arithmetic	ADD	R				ADDI{W[D] rd,rsl,rsl								Multiply Upper Half							R	MULHU rd,rsl,rsl,rsl					Compare Float <	R	FLT<(S[D]Q) rd,rsl,rsl	Store Quad SP	C	C.SQ rsl',rsl',imm																																																																																																																																																																																																																																																																																										
	ADD Immediate	R				ADDI{W[D] rd,rsl,rsl					Multiply Half Sign/Uns						R				MULHSU rd,rsl,rsl,rsl	Compare Float ≤									R	FLE(S[D]Q) rd,rsl,rsl	Float Store Word	CSS	C.FSWSP rsl,imm																																																																																																																																																																																																																																																																																							
	SUBTRACT	R		SUBI{W[D] rd,rsl,rsl		Multiply upper Half Uns			R								MULHU rd,rsl,rsl,rsl				Categorize													Classify Type	R	FCCLASS(S[D]Q) rd,rsl	Float Store Double SP	CSS	C.FSDSP rd,imm																																																																																																																																																																																																																																																																																			
	Load Upper Imm	I		LUI rd,imm	Divide		Divide	R	DIV{W[D] rd,rsl,rsl,rsl	Move			Move from Integer		R	FMV.S.X rd,rsl	Arithmetic	ADD	CR	C.ADD rd,rsl																																																																																																																																																																																																																																																																																																						
	ADD Upper Imm to PC	I	AUIPC rd,imm	Divide Unsigned			R	DIVU rd,rsl,rsl,rsl	Move to Integer			R						FMV.X.S rd,rsl	Convert	Convert from Int			R	FCVT.(S[D]Q).W rd,rsl	ADD Word	CR												C.ADDW rd',rsl',imm																																																																																																																																																																																																																																																																																				
Logical	XOR	R	XOR rd,rsl,rsl,rsl				Remainder	REMAINDER						R												REM{W[D] rd,rsl,rsl,rsl	Convert to Int Unsigned	R	FCVT.W.(S[D]Q) rd,rsl	Convert to Int								R	FCVT.WU.(S[D]Q) rd,rsl	ADD SP Imm * 16	CI	C.ADDI16SP x0,imm																																																																																																																																																																																																																																																																																
	XOR Immediate	R	XORI rd,rsl,rsl,rsl					Optional Atomic Instruction Extension: RVA			Category			Name								Fmt				RV32[64]128JA (Atomic)					Configuration Read	Read Rounding Mode	R								FRM rd	ADD SP Imm * 4	CI	C.ADDI4SPN rd',imm																																																																																																																																																																																																																																																																														
	OR	R	ORI rd,rsl,rsl,rsl			Load															Load Reserved													R	LR.{W[D]Q} rd,rsl	Read Flags	R						FRFLAG rd	Load Immediate	CI	C.LI rd,imm																																																																																																																																																																																																																																																																												
	OR Immediate	R	ORI rd,rsl,rsl,imm		Store					Store Conditional			R		SC.{W[D]Q} rd,rsl,rsl,rsl	Swap	SWAP																												R	AMOSWAP.{W[D]Q} rd,rsl,rsl,rsl	Load Upper Imm	CI	C.LUI rd,imm																																																																																																																																																																																																																																																																									
	AND	R	ANDI rd,rsl,rsl,rsl	Add					ADD			R						AMOADD.{W[D]Q} rd,rsl,rsl,rsl	Swap Status Reg	R			FSCSR rd,rsl,rsl	MeV	CR																							C.MV rd,rsl																																																																																																																																																																																																																																																																										
AND Immediate	R	ANDI rd,rsl,rsl,imm	Logical				XOR																		R		AMOXOR.{W[D]Q} rd,rsl,rsl,rsl	Swap Rounding Mode	R	FSRM rd,rsl,rsl								SUB	CR	C.SUB rd',rsl',imm																																																																																																																																																																																																																																																																																		
Compare	Set <	R						SLT rd,rsl,rsl,rsl			AND			AND								R				AMOAND.{W[D]Q} rd,rsl,rsl,rsl					Swap Flags	R	FSFLAG rd,rsl,rsl						SUB Word	CR	C.SUBW rd',rsl',imm																																																																																																																																																																																																																																																																																	
	Set < Immediate	R				SLTI rd,rsl,rsl,imm		OR													AMOR.													R	AMOR.{W[D]Q} rd,rsl,rsl,rsl	Swap Rounding Mode Imm	I			FSRWI rd,imm	Logical	XOR	CR	C.XOR rd',rsl',imm																																																																																																																																																																																																																																																																														
	Set < Unsigned	R			SLTU rd,rsl,rsl,rsl	Min/Max				MINIMUM			R		AMOMIN.{W[D]Q} rd,rsl,rsl,rsl	Swao Flags Imm	I																									FSFLAGI rd,imm	OR	CR	C.OR rd',rsl',imm																																																																																																																																																																																																																																																																													
	Set < Imm Unsigned	R		SLTIU rd,rsl,rsl,imm	MAXIMUM				MAXIMUM			R						AMOMAX.{W[D]Q} rd,rsl,rsl,rsl	Convert to Int Unsigned	R			FCVT.(L,T)U.(S[D]Q) rd,rsl	AND																				CR	C.AND rd',rsl',imm																																																																																																																																																																																																																																																																													
	Branches	Branch =	SB	BQ rsl,rsl,rsl,imm			MINIMUM Unsigned																		R		AMOMINU.{W[D]Q} rd,rsl,rsl,rsl	Convert to Int Unsigned	R	FCVT.(L,T)U.(S[D]Q) rd,rsl								AND Immediate						CB	C.ANDI rd',rsl',imm																																																																																																																																																																																																																																																																													
Branch <		S	BNE rsl,rsl,rsl,imm	MAXIMUM Unsigned							R			AMOMAXU.{W[D]Q} rd,rsl,rsl,rsl								3 Optional FP Extensions: RV64{128}{FIDQ}				Category					Name	Fmt	RV(FIDQ) (HP/SP,DP,QP)						Shifts					Shift Left Imm	CI	C.SLLI rd,imm																																																																																																																																																																																																																																																																												
Branch >		S	BLT rsl,rsl,rsl,imm					Jump & Link Register													J&L													J&L rd,rsl,imm	Move to Integer	R	FMV.X.(D)Q rd,rsl			Convert from Int	R			FCVT.(S[D]Q).(L,T)U rd,rsl	Shift Right Immediate	CB	C.SRLI rd',imm																																																																																																																																																																																																																																																																											
Branch >=		S	BLE rsl,rsl,rsl,imm			Jump				JUMP			JUMP rd,rsl,imm		Move from Integer	R	FMV.D.(D)X rd,rsl																									Convert to Int	R			FCVT.(S[D]Q).(L,T)U rd,rsl	Shift Right Arithmetic Imm	CB	C.SRAI rd',imm																																																																																																																																																																																																																																																																									
Branch < Unsigned		S	BLTU rsl,rsl,rsl,imm		Jump & Link Register				J&L			J&L rd,rsl,imm						Move to Integer	R	FMV.X.(D)Q rd,rsl			Convert to Int	R																								FCVT.(S[D]Q).(L,T)U rd,rsl	Branch=0	CB	C.BEQ0 rsl',imm																																																																																																																																																																																																																																																																							
Branch >= Unsigned	S	BGEU rsl,rsl,rsl,imm	Jump				JUMP																		JUMP rd,rsl,imm		Move to Integer	R	FMV.X.(D)Q rd,rsl	Convert to Int								R												FCVT.(S[D]Q).(L,T)U rd,rsl	Branch=0	CB	C.BNEZ0 rsl',imm																																																																																																																																																																																																																																																																					
Jump & Link	J&L	J&L		J&L rd,rsl,imm							Jump & Link Register			J&L								J&L rd,rsl,imm				Move to Integer					R	FMV.X.(D)Q rd,rsl	Convert to Int						R													FCVT.(S[D]Q).(L,T)U rd,rsl	Branch=0	CB	C.BNEZ0 rsl',imm																																																																																																																																																																																																																																																																			
	Jump & Link Register	J&L		J&L rd,rsl,imm				Jump													JUMP													JUMP rd,rsl,imm	Move to Integer	R	FMV.X.(D)Q rd,rsl			Convert to Int	R			FCVT.(S[D]Q).(L,T)U rd,rsl	Branch=0									CB	C.BNEZ0 rsl',imm																																																																																																																																																																																																																																																																			
	Synch	Synch thread		I		FENCE				Jump & Link Register			J&L		J&L rd,rsl,imm	Move to Integer	R																									FMV.X.(D)Q rd,rsl	Convert to Int			R	FCVT.(S[D]Q).(L,T)U rd,rsl							Branch=0	CB	C.BNEZ0 rsl',imm																																																																																																																																																																																																																																																																		
		Synch Instr & Data		I	FENCE.I	Jump & Link Register			J&L			J&L rd,rsl,imm						Move to Integer	R	FMV.X.(D)Q rd,rsl			Convert to Int	R																								FCVT.(S[D]Q).(L,T)U rd,rsl	Branch=0						CB	C.BNEZ0 rsl',imm																																																																																																																																																																																																																																																																		
		System	System CALL	I	SCALL		Jump & Link Register																		J&L		J&L rd,rsl,imm	Move to Integer	R	FMV.X.(D)Q rd,rsl								Convert to Int												R	FCVT.(S[D]Q).(L,T)U rd,rsl				Branch=0	CB	C.BNEZ0 rsl',imm																																																																																																																																																																																																																																																																	
System BREAK			I	SBREAK	Jump & Link Register						J&L			J&L rd,rsl,imm								Move to Integer				R					FMV.X.(D)Q rd,rsl	Convert to Int	R						FCVT.(S[D]Q).(L,T)U rd,rsl													Branch=0	CB			C.BNEZ0 rsl',imm																																																																																																																																																																																																																																																																		
Counters			Read CYCLE	I				RDWCYCLE rd													Jump & Link Register													J&L	J&L rd,rsl,imm	Move to Integer	R			FMV.X.(D)Q rd,rsl	Convert to Int			R	FCVT.(S[D]Q).(L,T)U rd,rsl								Branch=0			CB	C.BNEZ0 rsl',imm																																																																																																																																																																																																																																																																	
	Read CYCLE upper Half		I	RDWCYCLEH rd				Jump & Link Register		J&L			J&L rd,rsl,imm		Move to Integer	R	FMV.X.(D)Q rd,rsl																									Convert to Int	R			FCVT.(S[D]Q).(L,T)U rd,rsl	Branch=0							CB		C.BNEZ0 rsl',imm																																																																																																																																																																																																																																																																		
	Read Time		I	RDTIME rd		Jump & Link Register			J&L			J&L rd,rsl,imm						Move to Integer	R	FMV.X.(D)Q rd,rsl			Convert to Int	R																								FCVT.(S[D]Q).(L,T)U rd,rsl	Branch=0					CB		C.BNEZ0 rsl',imm																																																																																																																																																																																																																																																																		
	Read Time upper Half	I	RDTIMEH rd	Jump & Link Register			J&L																		J&L rd,rsl,imm		Move to Integer	R	FMV.X.(D)Q rd,rsl	Convert to Int								R												FCVT.(S[D]Q).(L,T)U rd,rsl	Branch=0			CB	C.BNEZ0 rsl',imm																																																																																																																																																																																																																																																																			
	Read INSTR RETired	I	RDINSTRET rd		Jump & Link Register						J&L			J&L rd,rsl,imm								Move to Integer				R					FMV.X.(D)Q rd,rsl	Convert to Int	R						FCVT.(S[D]Q).(L,T)U rd,rsl													Branch=0		CB	C.BNEZ0 rsl',imm																																																																																																																																																																																																																																																																			
Read INSTR upper Half	I	RDINSTRETH rd	Jump & Link Register																		J&L													J&L rd,rsl,imm	Move to Integer	R	FMV.X.(D)Q rd,rsl			Convert to Int	R			FCVT.(S[D]Q).(L,T)U rd,rsl	Branch=0								CB	C.BNEZ0 rsl',imm																																																																																																																																																																																																																																																																				
CI	15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0	R						R		R			R		R	R	R																									R	R			R	R						R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R

“A”: Atomic Operations Extension

- Two classes:
- Atomic Memory Operations (AMO)
 - Fetch-and-op,
op=ADD,OR,XOR,MAX,MIN,MAXU,MINU
- Load–Reserved/Store Conditional
 - With forward progress guarantee for short sequences
- All atomic operations can be annotated with two bits (Acquire/Release) to implement release consistency or sequential consistency



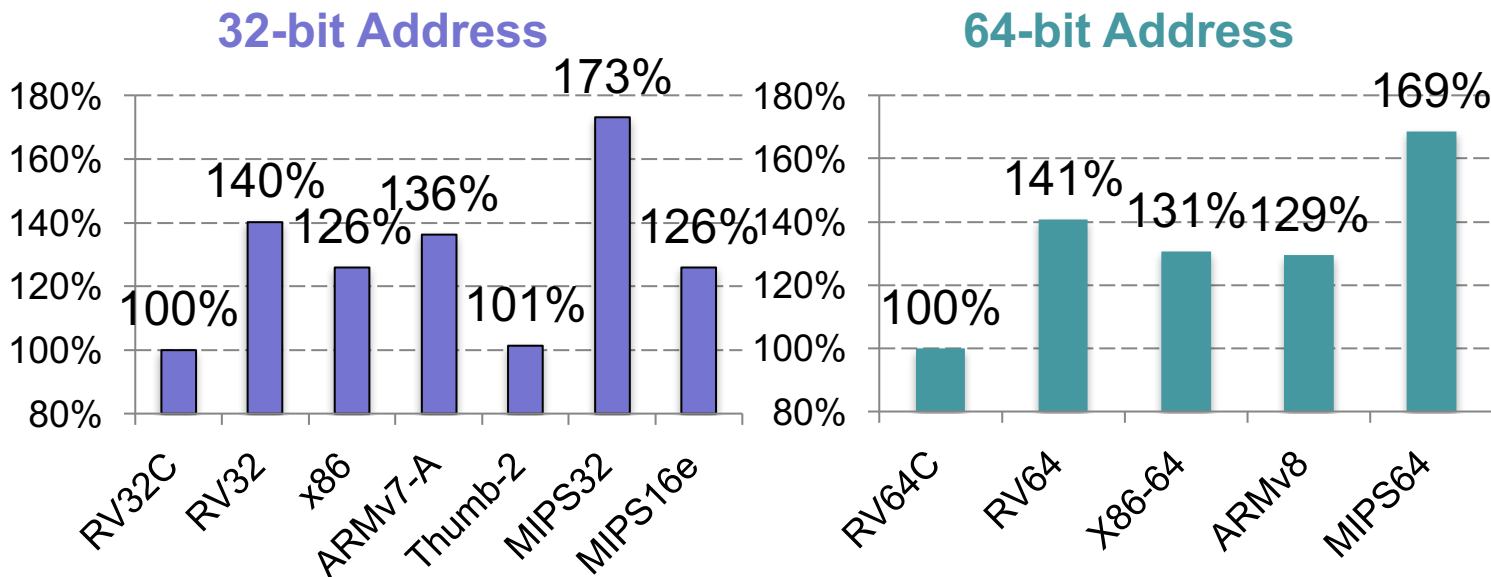
“C”: Compressed Instruction Extension

- Compressed code important for:
 - low-end embedded to save static code space
 - high-end commercial workloads to reduce cache footprint
- C extension adds 16-bit compressed instructions
 - 2-address forms with all 32 registers
 - 2/3-address forms with most frequent 8 registers
- 1 compressed instruction expands to 1 base instruction
 - Assembly language programmer & compiler oblivious
 - RVC $\Rightarrow$ RVI decoder only ~ 700 gates ($\sim 2\%$ of small core)
- All original 32-bit instructions retain encoding but now can be 16-bit aligned
- 50%-60% instructions compress $\Rightarrow$ 25%-30% smaller



SPECint2006 compressed code size with save/restore optimization (relative to “standard” RVC)

- RISC-V now smallest ISA for 32- and 64-bit addresses





Goals for RISC-V Standard Vector Extension

- Efficient and scalable to all reasonable design points
 - Low-cost or high-performance
 - In-order, decoupled, or out-of-order microarchitectures
 - Integer, fixed-point, and/or floating-point data types
- Good compiler target
- Support both implicit auto-vectorization (OpenMP) and explicit SPMD (OpenCL) programming models
- Work with virtualization layers
- Fit into standard fixed 32-bit encoding space
- Be base for future vector++ extensions

- Non-goal: Look like everyone's Packed-SIMD
- Non-goal: Look like a GPU

RISC-V Privileged Architecture

Application
ABI
AEE

Application	Application
ABI	ABI
OS	
SBI	
SEE	

Application	Application	Application	Application
ABI	ABI	ABI	ABI
OS		OS	
SBI		SBI	
Hypervisor			
HBI			
HEE			

- Provide clean split between layers of the software stack
- Application communicates with Application Execution Environment (AEE) via Application Binary Interface (ABI)
- OS communicates via Supervisor Execution Environment (SEE) via System Binary Interface (SBI)
- Hypervisor communicates via Hypervisor Binary Interface to Hypervisor Execution Environment
- All levels of ISA designed to support virtualization

RISC-V Privilege Modes

- Four privilege modes
 - User (U-mode)
 - Supervisor (S-mode)
 - Hypervisor (H-mode)
 - Machine (M-mode)
- Supported combinations of modes:
 - M (simple embedded systems)
 - M, U (embedded systems with protection)
 - M, S, U (systems running Unix-style operating systems)
 - M, H, S, U (systems running Hypervisors)

Supervisor Binary Interface

- Platform-specific functionality abstracted behind SBI
 - Query physical memory map
 - Enumerate devices
 - Get hardware thread ID and # of hardware threads
 - Save/restore coprocessor state
 - Query timer properties, set up timer interrupts
 - Send interprocessor interrupts
 - Send TLB shootdowns
 - Reboot/shutdown
- Simplifies hardware acceleration
- Simplifies virtualization
- Draft SBI to be released with next priv. ISA draft

Summarizing - Why RISC-V?

- Free and open architecture, no proprietary lock-in
- Much simpler ISA than others
 - Verification, security
- Readily and freely extensible
 - Reduce need for application/accelerator split
- Usable as base ISA for every core on SoC
 - Reduces software complexity



Outline

- RISC-V ISA Background
- RISC-V Foundation Overview
 - Foundation Structure
 - Membership Update
- SW & HW Ecosystem
- Summary





Foundation Mission Statement

The RISC-V Foundation is a non-profit consortium chartered to standardize, protect, and promote the free and open RISC-V instruction set architecture together with its hardware and software ecosystem for use in all computing devices.



RISC-V is NOT an Open-Source Processor

- RISC-V is an ISA specification – NOT an open-source processor core
- Most of the cost of chip design is in software, so the goal is to make sure software can be reused across many chip designs
- The Foundation encourages both open-source and proprietary implementations of the RISC-V ISA specification

Foundation Principles

- The RISC-V ISA and related standards shall remain open and license-free to all parties. The standard specifications shall always be publicly available as an online download
- The compatibility test suites shall always be publicly available as a source code download
- To protect the standard, only members (with commercial RISC-V products) of the Foundation in good standing can use “RISC-V” and associated trademarks, and only for devices that pass the tests in the open-source compatibility suites maintained by the Foundation

Foundation Function

- official source of information about RISC-V, maintains an online repository of RISC-V documents, and promotes adoption of RISC-V by organizing both online and live events
- responsible for sustaining, evolving and open-source licensing of the RISC-V instruction set architecture and surrounding hardware and software ecosystem over time in response to changes in technology and the needs and requests of the user community
- manages licensing of the RISC-V trademarks and provides a vehicle to decide whether a project or product can use the RISC-V trademark
- maintains a directory of public-domain instruction set architecture and micro-architectural techniques, culled from publications and expired patents
- produces and sells RISC-V promotional material, the proceeds from which are used to further the goals of the Foundation

Foundation Organization

- The Board of Directors consists of seven+ members, whose replacements are elected by the membership
- The Board is ultimately responsible for fulfilling the Mission Statement
- The Board can amend the By-Laws of the RISC-V foundation via a two-thirds affirmative vote
- The Board appoints chairs of ad-hoc committees to address issues concerning RISC-V, and has the final vote of approval of the recommendation of the ad-hoc committees.
- All members of committees must be members of the RISC-V Foundation
 - Committee chairs must report to the Board, and committees are subject to termination if the Board decides a committee is not making satisfactory progress
- Given the worldwide interest in RISC-V, the Board will appoint Chairs of Regional Committees to promote RISC-V development in local communities

Classes of Membership

- Platinum Sponsor dues of US\$25,000 per year
 - 10 complimentary registrations for RISC-V meetings during the year of membership and the most prominent and largest display of company logos in online and print materials for RISC-V
- Platinum members are eligible for Board seat elections and to Chair Foundation Technical Committees, Marketing Committees and Sub-Committees
- Gold Sponsor dues of US\$10,000 per year
 - 7 complimentary registrations for RISC-V meetings during the year of membership and prominent display of company logos in online and print materials for RISC-V
- Gold Sponsor members are eligible to Chair Foundation Technical Committees, Marketing Committees and Sub-Committees
- Silver Sponsor dues of US\$5,000 per year
 - 5 complimentary registrations for RISC-V meetings and for their logos to be displayed in RISC-V online and print materials
- All Sponsor organizations have one vote per open position in Board elections.
- Auditor dues of US\$2,500 per year (non-voting)
 - 2 complimentary registrations for RISC-V meetings during the year of membership and for their names to be listed in online and print materials for RISC-V
- Individual Member dues of US\$99 per year (non-voting)

Foundation Status

- Articles of Incorporation filed August, 2015
 - RISC-V Foundation Corporation is a legal, non-profit operating entity – 501(c)(6)
- 51 members have joined the Foundation
- Board of Directors formed in Q2, 2016
- Technical & Marketing Committees formed in Q3, 2016
 - Chairs & Vice-Chairs elected
- Membership Agreement & Bylaws ratified and released in Q4, 2016

Membership Agmt Key Terms

- Free and open license – Foundation grants to the Member a non-exclusive, zero royalty right and license ...to use, make, have made, offer for sale, sell, export, import, practice, copy, ...the RISC-ISA as then constituted (the "Member License")
- Member Contributions & Improvements – Member grants to the Foundation a non-exclusive, zero royalty right and license ...to use, make, have made, offer for sale, sell, export, import, practice, copy, ...for each and all of the Improvements and each and all Intellectual Property Rights in and to such Improvements (the "Core License")

Bylaws Key Terms

- Defines the role of the Foundation
- Defines Membership Levels and Rights
 - One vote per company
- Defines the role of the Board of Directors
 - Ratify all Committee recommendations (Technical, Marketing, etc)
- Defines Committee Responsibilities
 - All specification development to be done by Membership Committees
 - All specification development must include at least one round of public consultation (outside the membership)

RISC-V Foundation Board of Directors

- **Krste Asanović, Chairman**
 - Professor in the EECS Department at UC Berkeley
- **Zvonimir Bandić**
 - Senior Director of Next Generation Platform Technologies at Western Digital Corporation
- **Charlie Hauck**
 - CEO of Bluespec Inc.
- **David Patterson**
 - Retired Professor Computer Science UC Berkeley, now with Google
- **Jothy Rosenberg**
 - Associate Director of the cyber security group at Draper Laboratories
- **Frans Sijstermans**
 - Vice President Engineering at NVIDIA
- **Ted Speers**
 - Technical Fellow, Head of Product Architecture for Microsemi's SoC Group



Berkeley
Architecture
Research



**Western
Digital®**



DRAPER



NVIDIA®



Technical Committee

- Chair Yunsup Lee, SiFive, Vice Chair Silviu Chiricescu, BAE Systems
- Currently consists of 10 Task Groups:
 - Opcode space management, Krste Asanovic, UC Berkeley
 - Privileged ISA specification, Andrew Waterman, SiFive
 - Memory model, Dan Lustig, NVIDIA
 - “B”it manipulation extension, Rex McCrary, AMD
 - Formal specification, Rishiyur Nikhil, Bluespec
 - Debug specification, Megan Wachs, SiFive
 - Security, Joe Xie, NVIDIA
 - J standard extension, David Chisnall, Cambridge
 - Vector extensions, Krste Asanovic, UC Berkeley
 - SW tool chain, Arun Thomas, BAE Systems

RISC-V Marketing Task Groups

- Chair Arun Thomas, BAE Systems, Vice-Chair Ted Marena, Microsemi
- Currently consists of 5 Task Groups:
 - RISC-V Content, Michael Gielda, Antmicro
 - Generate new content to increase RISC-V awareness
 - Workshops, Jack Kang, SiFive
 - Increase the profile of RISC-V Workshops
 - Industry Outreach, Neil Hand, Codaip
 - Increase RISC-V participation in external industry events
 - Open Source and University Outreach, Arun Thomas, BAE
 - Outreach to the open-source community and universities
 - RISC-V 2020, Art Swift, Esperanto
 - Track RISC-V adoption, spot future trends, and consider RISC-V's future direction



Berkeley
Architecture
Research

Platinum Members

bluespec

cortus

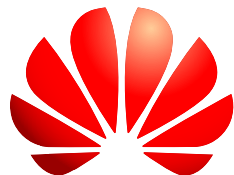
DRAPER

IBM

Google



Hewlett Packard
Enterprise



HUAWEI

Micron



Mellanox
TECHNOLOGIES



Microsemi



Microsoft



NVIDIA

NXP

Rambus

QUALCOMM

Cryptography Research



SiFive



Western
Digital

Gold, Silver & Auditor Members



Some Workshop Stats...

- 1st RISC-V workshop Jan 14-15, 2015 in Monterey, CA
 - Sold out: 144 (33 companies & 14 universities)
 - [Slides & videos can be found here](#)
- 2nd RISC-V workshop Jun 29-30, 2015 at UC Berkeley, CA
 - Sold out: 120 (30 companies & 20 universities)
 - [Slides & videos can be found here](#)
- 3rd RISC-V workshop Jan 5-6, 2016 at Oracle Redwood City, CA
 - Sold out: 157 (42 companies & 26 universities)
 - [Slides & videos can be found here](#)
- 4th RISC-V workshop Jul 12-13, 2016 at MIT Cambridge, MA
 - Sold out: 252 (63 companies & 42 universities)
 - [Slides & videos can be found here](#)
- 5th RISC-V workshop Nov 29-30, 2016 at Google Mountain View, CA
 - Sold out: 350+ (107 companies & 29 universities)
 - [Slides & videos can be found here](#)

5th RISC-V Workshop – Thanks Google!!



- Sold out! People turned away...
- 100+ companies, 350+ attendees

Nov 29th – 30th, 2016
Google Mountain View, CA





6th RISC-V Workshop – Registration Open

May 8th – 11th , 2017 Shanghai China

[Registration & Call for Papers details](#)



nVIDIA®





Outline

- RISC-V ISA Background
- RISC-V Foundation Overview
 - Foundation Structure
 - Membership Update
- SW & HW Ecosystem
- Summary





SW Ecosystem



RISC-V Emulation / Simulation

- Spike - RISC-V ISA simulator (riscv-isa-sim)
- QEMU - Full system and user emulation
- RISCVEMU - Boot RISC-V/Linux in your browser
- gem5 (in progress) - Full-system simulator

- Binutils
 - RISC-V support will be in 2.28 release
- GCC
 - Steering Committee accepted port for inclusion. Port will likely make it into 7.1
- LLVM (in progress)
 - Some patches upstream. Codegen patches in progress.

RISC-V OS and Firmware

- Firmware: coreboot and UEFI
- Linux: Fedora, Debian, Gentoo, Yocto/Poky
- BSD: FreeBSD, NetBSD
- Other OS: seL4, Genode, HelenOS, Zephyr, FreeRTOS, RTEMS, MyNewt
- FreeBSD and Zephyr have upstream RISC-V support

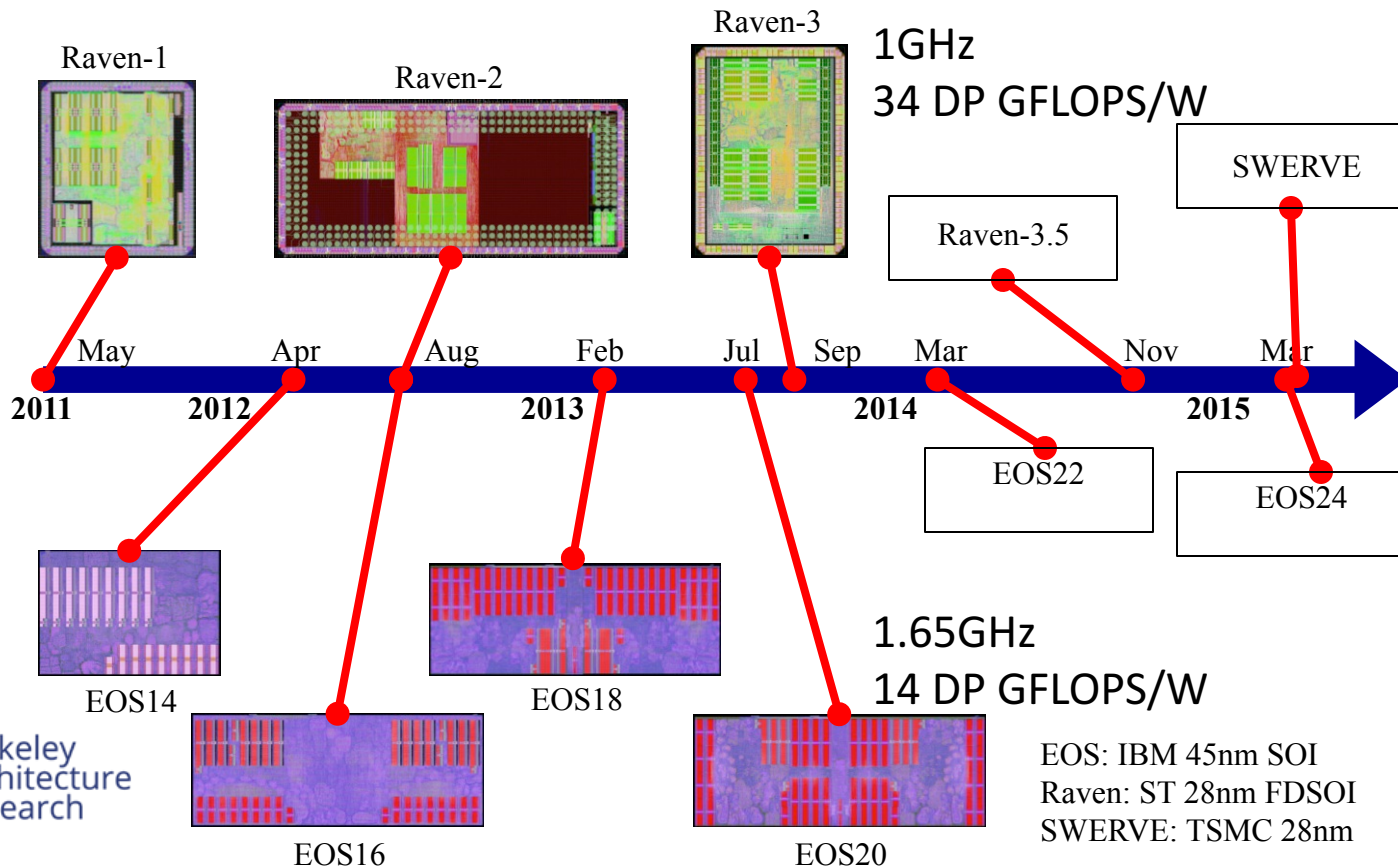


HW Ecosystem

RISC-V Hardware

- RISC-V cores/SoCs for many different use cases
 - Education, research, commercial products
 - Small, low-cost microcontrollers to high-performance multicore chips
- Written in a variety of HDLs
 - VHDL, Verilog, Chisel, Bluespec SystemVerilog

Five 28nm & Six 45nm RISC-V Chips (11 Tapeouts So Far)



UC Berkeley Hardware

- System on Chip (SoC)
 - Rocket Chip - Parameterized RISC-V SoC generator
- Cores
 - Rocket Core - 5 stage pipeline, single-issue
 - BOOM - Out-of-order core
 - Sodor - Educational cores (1-5 stage)
- <https://github.com/ucb-bar>



UC Berkeley and SiFive

- UC Berkeley Architecture Research (UCB BAR) group created RISC-V
- UCB BAR designed several open source cores/SoCs for research and teaching
- RISC-V creators formed a startup (SiFive) to design custom RISC-V chips for customers
 - Building on their open source cores



Berkeley
Architecture
Research



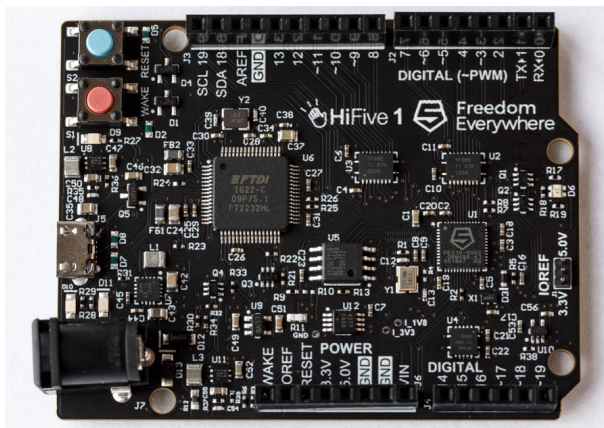
SiFive

SiFive Hardware

- Building on Rocket Core and Rocket Chip SoC Gen
- **Freedom Everywhere:** Low cost, 32-bit microcontrollers
- **Freedom Unleashed:** High performance, 64-bit multi-core SoCs
- <https://github.com/sifive>



SiFive HiFive1



- First generally available open-source RISC-V silicon
- Freedom Everywhere 310
- Arduino Compatible
- Supports Arduino IDE
- \$59 US from Crowd Supply

<https://www.crowdsupply.com/sifive/hifive1>



lowRISC

- Based at the University of Cambridge
- Aiming to build a low cost development board.
“Raspberry Pi for Grownups”
- Builds on Rocket Chip SoC Generator
- Tagged architecture and minion cores
- See Alex Bradbury’s FOSDEM 2015 talk
- <https://github.com/lowrisc>



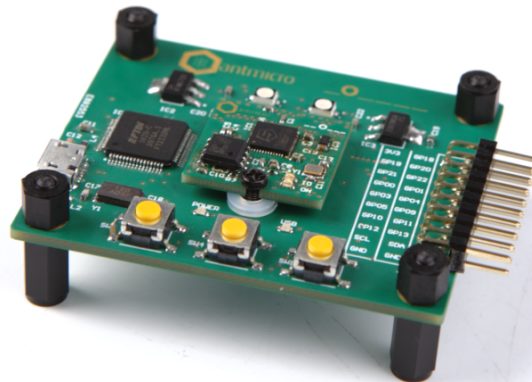
RISC-V Shakti Project at IIT Madras

- RISC-V is the "standard ISA" for India
- IIT Madras building 6 open-source cores, from microcontrollers to supercomputers
- Using Bluespec SystemVerilog as HDL
- See https://bitbucket.org/casl/shakti_public



Antmicro RISC-V SoM

- RISC-V SoM based on SiFive's Freedom E310
- complementary baseboard
- runs Zephyr RTOS
- open source HW & SW



Codasip Application optimized RISC-V

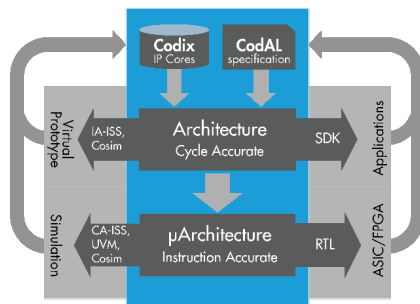


Codix-Bk Processor IP

- RISC-V ISA compliant
- Codix-Bk3: 3-stage 32 bit
- Codix-Bk5: 5-stage 32-bit or 64 bit
- Support for standard and custom extensions
- Configurable registers, Interrupts, and Branch Prediction Unit
- Optional instruction and data cache
- Optional MMU/MPU
- Full JTAG and debug support
- Sleep modes and low power support



Codasip Studio processor design and customization environment

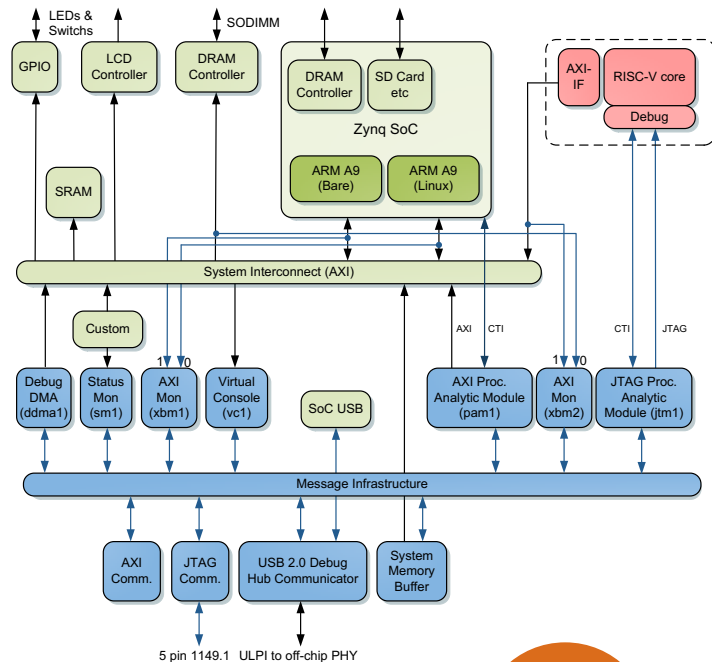
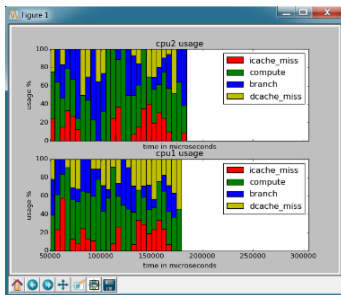
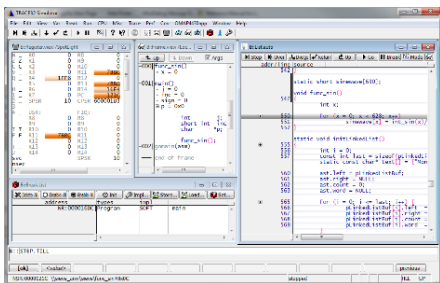


- Profile target RISC-V applications
- Explore optimization opportunities, and implement extensions
- Automatically regenerate RTL and LLVM-based SDK
- Comprehensive verification and simulation



UltraSoC RISC-V Debug & Analytics

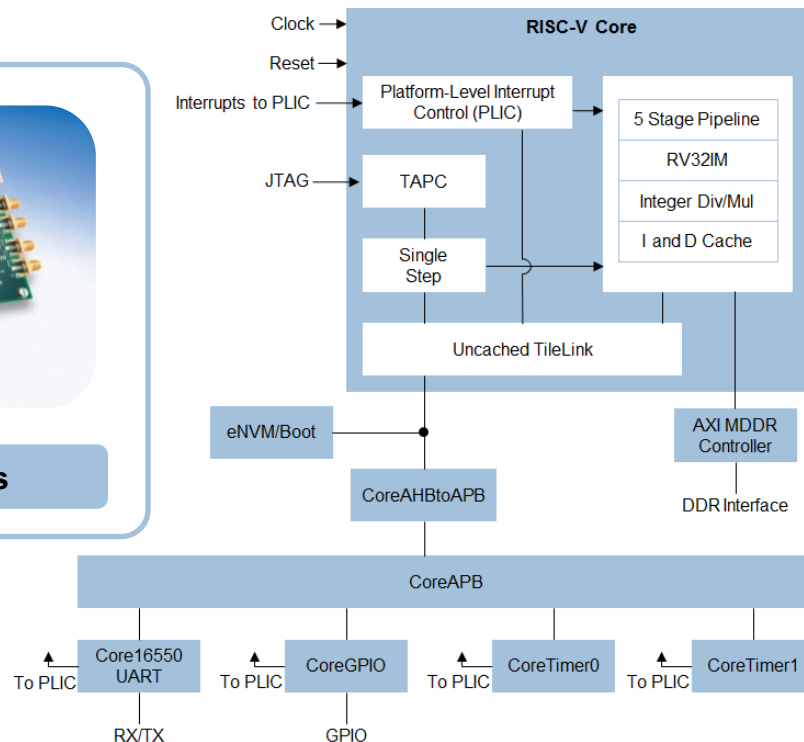
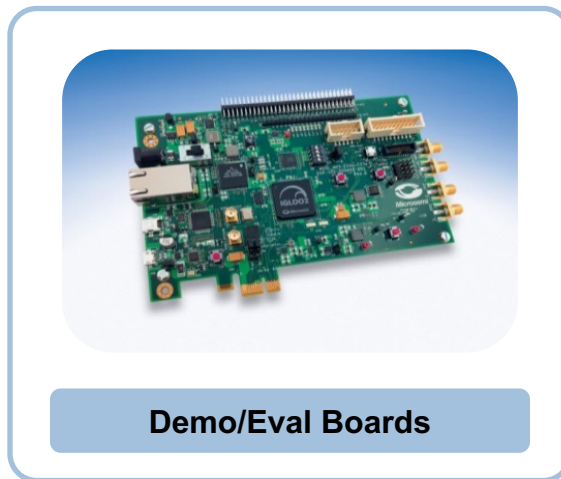
- Debug, analyze, optimize
- Full support for run-control, trace etc
- Full support for RISC-V and other processors
- SoC-wide analytics
- Connectivity options: JTAG, USB, high-speed serial, etc
- Support for Eclipse/GDB and third-party tools
- Silicon proven



ultra soc

Microsemi RISC-V FPGA Soft CPU

- 5 stage pipeline
- 8K I/D cache
- Integer mul/div
- 8 break points
- 31 interrupts
 - Priority by Int #
- 1.1 DMIPS/MHz
- Boot from eNVM
- Boot from SPI flash
- 12K LE's
- 100 MHz-150 MHz



Syntacore SCRx RISC-V Cores

- SCRx: a family of the state-of-the art RISC-V compatible synthesizable processor cores
 - SCR1: RV32IC[EM]
 - SCR3: RV32IMC[E]
 - SCR4: RV32IMCF[D]
 - SCR5: RV[32|64]IMC[AFD]
- Stable, configurable, available for evaluation
- Baseline: every core can be extended/customized



Syntacore™
Custom cores and tools

Bluespec 'Flute' RISC-V Core

- RV32/64 I, M, F, D
- Boots Linux
- 6-stage in-order pipeline
 - Branch prediction
 - Concurrent integer, floating point, memory pipes
- “Uncore” components:
 - MMUs, L1 caches, L2 caches
- SoC components:
 - Interconnect fabric, memory controller, DMA engine, with easy expandability and substitutability to incorporate your own components



VectorBlox RISC-V ORCA +LVE

ORCA RISC-V family is free, portable, FPGA-optimized for low cost and low power



	VectorBlox ORCA RV32IM	+ LVE
Area	2,900 LUTs 20 BRAMs	3,800 LUTs 20 BRAMs + scratchpad
Clock speed	17 MHz	17 MHz
FIR filter (32 taps)	15 cycles/tap	1.25 cycles/tap 12x speedup

Gray Research RISC-V GRVI Phalanx

- GRVI (“Groovy”) Gray Research RISC-V RV*I
- Purpose: efficient parallel processing element
- Scalar, 2-3 stage pipeline RV32I+- + MUL/H^{opt}
 - 300-375 MHz (KU-2), 1.3-1.6? CPI
 - ~320 6-LUTs
 - ~1 “MIPS”/LUT



Other RISC-V SoCs / Cores

- PULPino (ETH Zurich) <https://github.com/pulp-platform/pulpino>

ETH zürich



- PicoRV32 <https://github.com/cliffordwolf/picorv32>
- YARVI <https://github.com/tommythorn/yarvi>
- Many, many more RISC-V cores
 - Both commercial and open source
 - <https://github.com/riscv/riscv-wiki/wiki/RISC-V-Cores-and-SoCs>

Summary: Why RISC-V?

- Free and open architecture, no proprietary lock-in
- Much simpler ISA than others (verification, security)
- Stable, will not change or disappear
- Enables better area/power/performance than other general-purpose ISAs at all design points
- Usable as base ISA for every core on SoC
- Readily and freely extensible

Modest RISC-V Project Goal

*Become the industry-standard ISA for
all computing devices*

RISC-V Foundation Summary

- Strong Industry Support
 - 51 member companies to date
 - Broad commercial and academic interest (sold out 350+ 5th workshop attendees representing 107 companies & 29 universities)
- Technical & Marketing Committees very active
- Strong and growing SW & HW ecosystem
- 6th RISC-V Workshop details

May 8th – 11th , 2017
Shanghai China

